

Demo Runbook 1

Introduction to MLOps • ML Lifecycle • Environment Setup • Model
Serialization & Configuration Management

Copy-paste-ready commands, expected outputs, and talking points for running each lab live as a demo.

How to use this	Run the Setup block once before the audience arrives. During the demo, execute each numbered Step live, pausing on the 'Expected output' box so the audience sees the result before you move on.
Total demo time	Approx. 25-35 minutes for all 4 labs

LAB 1

Introduction to MLOps

Contrasting a manual ML workflow with an automated, tracked one

Demo Goal: Show the audience the concrete difference between a manual, untracked training run and one wrapped with minimal MLOps tooling — in under 10 minutes.

Setup (run once before the demo)

```
mkdir mlops_intro_demo && cd mlops_intro_demo
python3 -m venv venv && source venv/bin/activate
pip install scikit-learn mlflow pandas
```

STEP 1

Run the 'manual' version first

Create `manual_train.py` — a plain script with no tracking, no saved metadata.

```
# manual_train.py
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
print('Accuracy:', accuracy_score(y_test, model.predict(X_test)))
```

Expected output / what you'll see:

Accuracy: 0.972...
(Note: the number, parameters, and model are gone the moment this script exits.)

Say to audience: "This works, but the moment this script finishes, we've lost the parameters, the metrics, and the model itself. If someone asks 'what accuracy did we get last Tuesday with what settings?' — we can't answer."

STEP 2

Run the same training with minimal MLOps tracking

Create `tracked_train.py` — the same logic, wrapped with three MLflow calls.

```
# tracked_train.py
import mlflow
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

mlflow.set_experiment('intro-demo')
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

with mlflow.start_run():
    n_estimators = 100
    mlflow.log_param('n_estimators', n_estimators)
    model = RandomForestClassifier(n_estimators=n_estimators)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_metric('accuracy', acc)
```

```
mlflow.sklearn.log_model(model, 'model')
print('Logged run with accuracy:', acc)
```

Expected output / what you'll see:

Logged run with accuracy: 0.972...
(This time, the run is permanently recorded.)

STEP 3

Open the MLflow UI and show the recorded run

```
mlflow ui    # open http://127.0.0.1:5000
```

Expected output / what you'll see:

The 'intro-demo' experiment appears with one run showing:
parameter n_estimators=100, metric accuracy=0.97+, and a downloadable model artifact.

Say to audience: *"That's the core idea of MLOps in one sentence: turn manual, invisible steps into automated, recorded, and reproducible ones. Everything else we build today is a variation on this same principle."*

Demo Wrap-up

Keep both scripts open side by side — this comparison is the strongest opening visual for the whole demo series, since every later lab builds on the 'tracked_train.py' pattern.

LAB 2

ML Lifecycle

Running a full ingest -> train -> evaluate -> register mini-pipeline end to end

Demo Goal: Show all five lifecycle stages executing as one connected flow, not five disconnected scripts.

Setup (run once before the demo)

```
pip install pandas scikit-learn mlflow
mkdir -p lifecycle_demo/src && cd lifecycle_demo
```

STEP 1

Write the pipeline stages as separate functions in one file

```
# src/pipeline.py
import mlflow, mlflow.sklearn
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

def ingest():
    X, y = load_wine(return_X_y=True)
    print(f'[ingest] Loaded {len(X)} rows')
    return X, y

def validate(X, y):
    assert X.shape[0] == y.shape[0], 'row mismatch'
    assert not (X != X).any(), 'NaNs found' # NaN check
    print('[validate] Data checks passed')

def train(X, y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = RandomForestClassifier(n_estimators=150).fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f'[train] Trained model, accuracy={acc:.3f}')
    return model, acc

def evaluate(acc, threshold=0.85):
    passed = acc >= threshold
    print(f'[evaluate] acc={acc:.3f} vs threshold={threshold} -> {"PASS" if passed else "FAIL"}')
    return passed

def register(model, acc):
    mlflow.set_experiment('lifecycle-demo')
    with mlflow.start_run():
        mlflow.log_metric('accuracy', acc)
        mlflow.sklearn.log_model(model, 'model', registered_model_name='wine-lifecycle-demo')
    print('[register] Model registered')

if __name__ == '__main__':
    X, y = ingest()
    validate(X, y)
    model, acc = train(X, y)
    if evaluate(acc):
        register(model, acc)
    else:
        print('Model did not meet quality bar – not registered.')
```

STEP 2

Run the full lifecycle live

```
python src/pipeline.py
```

Expected output / what you'll see:

```
[ingest] Loaded 178 rows
[validate] Data checks passed
[train] Trained model, accuracy=0.972
[evaluate] acc=0.972 vs threshold=0.85 -> PASS
[register] Model registered
```

Say to audience: "Notice the pipeline printed its way through every lifecycle stage in order, and only registered the model because it passed the quality gate. If I lower the threshold below the achievable accuracy artificially, watch what happens next."

STEP 3

Show the quality gate actually blocking a bad model

Temporarily edit the call to evaluate() to use an unreachable threshold, then re-run.

```
# change:  if evaluate(acc):
# to:      if evaluate(acc, threshold=0.999):
python src/pipeline.py
```

Expected output / what you'll see:

```
[evaluate] acc=0.972 vs threshold=0.999 -> FAIL
Model did not meet quality bar – not registered.
```

Say to audience: "This is the whole point of a lifecycle, not just a script: bad models get stopped automatically before they ever reach a registry or a user."

Demo Wrap-up

Revert the threshold back to 0.85 after the demo so subsequent runs register successfully again.

LAB 3

Environment Setup

Showing why a pinned, containerized environment prevents 'works on my machine'

Demo Goal: Demonstrate a reproducible environment using a requirements file and a Docker container, and show it running identically regardless of the host machine's Python setup.

Setup (run once before the demo)

```
mkdir env_setup_demo && cd env_setup_demo
```

STEP 1

Create a pinned requirements file

```
cat > requirements.txt << 'EOF'
scikit-learn==1.5.2
pandas==2.2.3
mlflow==2.16.2
EOF
```

STEP 2

Write a tiny script that prints the environment fingerprint

```
cat > check_env.py << 'EOF'
import sklearn, pandas, sys
print('Python:', sys.version.split()[0])
print('scikit-learn:', sklearn.__version__)
print('pandas:', pandas.__version__)
EOF
```

STEP 3

Write the Dockerfile

```
cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY check_env.py .
CMD ["python", "check_env.py"]
EOF
```

STEP 4

Build and run the container

```
docker build -t env-demo .
docker run --rm env-demo
```

Expected output / what you'll see:

```
Python: 3.11.x
scikit-learn: 1.5.2
pandas: 2.2.3
```

Say to audience: "This exact output is guaranteed no matter whose laptop, whose CI runner, or whose cloud VM this runs on — that's the entire value proposition of environment reproducibility."

STEP 5**(Optional live contrast) Run the same script directly on the host**

```
python3 check_env.py
```

Expected output / what you'll see:

Likely different version numbers than the container output above – proving the point that the host's ad hoc environment is not guaranteed to match.

Say to audience: *"Notice the versions may differ from the container run — this drift is exactly what causes bugs that 'only happen in production' or 'only happen on my machine.'"*

Demo Wrap-up

If Docker isn't available in the demo room, substitute 'pip freeze > requirements.lock.txt' and show the lockfile pinning every transitive dependency as a lighter-weight version of the same idea.

LAB 4

Model Serialization & Configuration Management

Saving/loading a trained model and moving hardcoded values into a config file

Demo Goal: Show a model being serialized, reloaded, and used for prediction — and contrast hardcoded hyperparameters with a clean external config file.

Setup (run once before the demo)

```
pip install scikit-learn joblib pyyaml
mkdir serialization_demo && cd serialization_demo
```

STEP 1

Train and serialize a model with joblib

```
cat > train_and_save.py << 'EOF'
import joblib
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)
model = RandomForestClassifier(n_estimators=100).fit(X, y)
joblib.dump(model, 'model.joblib')
print('Model saved to model.joblib')
EOF
python train_and_save.py
```

Expected output / what you'll see:

```
Model saved to model.joblib
```

STEP 2

Load the serialized model in a completely separate script and predict

```
cat > predict.py << 'EOF'
import joblib
from sklearn.datasets import load_wine

model = joblib.load('model.joblib')
X, y = load_wine(return_X_y=True)
print('Prediction for first sample:', model.predict(X[:1])[0])
print('Actual label:', y[0])
EOF
python predict.py
```

Expected output / what you'll see:

```
Prediction for first sample: 0
Actual label: 0
```

Say to audience: *"This is the key idea: the process that trains a model and the process that serves predictions can now be completely different scripts, machines, or services — connected only by this one file."*

STEP 3

Now replace hardcoded hyperparameters with a config file

```
cat > config.yaml << 'EOF'
model:
  n_estimators: 150
  max_depth: 8
EOF
```

```
    random_state: 42
EOF

cat > train_with_config.py << 'EOF'
import yaml, joblib
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier

cfg = yaml.safe_load(open('config.yaml'))['model']
print('Using config:', cfg)

X, y = load_wine(return_X_y=True)
model = RandomForestClassifier(**cfg).fit(X, y)
joblib.dump(model, 'model.joblib')
print('Model retrained and saved using external config.')
EOF
python train_with_config.py
```

Expected output / what you'll see:

```
Using config: {'n_estimators': 150, 'max_depth': 8, 'random_state': 42}
Model retrained and saved using external config.
```

STEP 4

Change one value in config.yaml and re-run without touching any code

Edit config.yaml — change `n_estimators` from 150 to 300 — then re-run the same command.

```
python train_with_config.py
```

Expected output / what you'll see:

```
Using config: {'n_estimators': 300, 'max_depth': 8, 'random_state': 42}
```

Say to audience: "Zero code changes were needed to retrain with completely different settings. This is what separates configuration from code — and it's what lets non-engineers (or automated tuning jobs) safely change model behavior."

Demo Wrap-up

This lab connects directly to the Model Registry and Hyperparameter Optimization demos — the same config.yaml pattern is what a hyperparameter sweep tool would generate automatically.